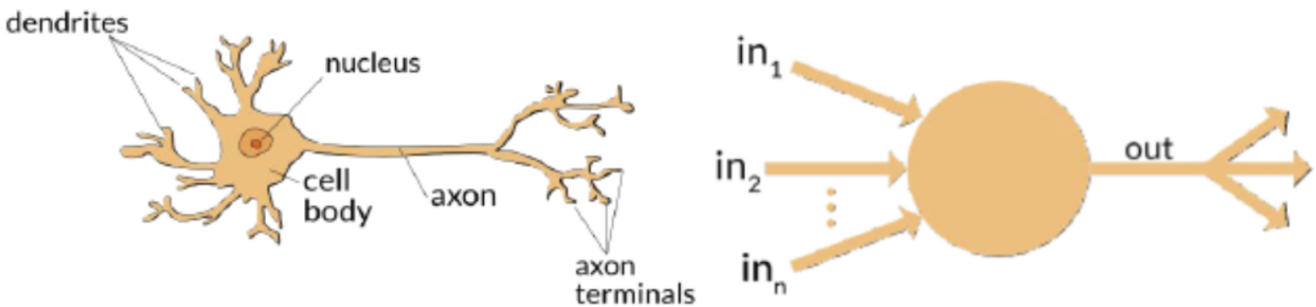
COMP7606:
Deep Learning
Intro to Artificial Neural Networks [1]

Mauro Sozio

[1] some slides prepared with the help of Beth Chan and Francis Chin

The Perceptron

- Developed in 1958 by F. Rosenblatt [2].
- Inspired by neurobiology (as airplanes are inspired by bird)
- Neurons receiving "enough" electric signal from other neurons get "activated" and "fire" an electric signal to other neurons.

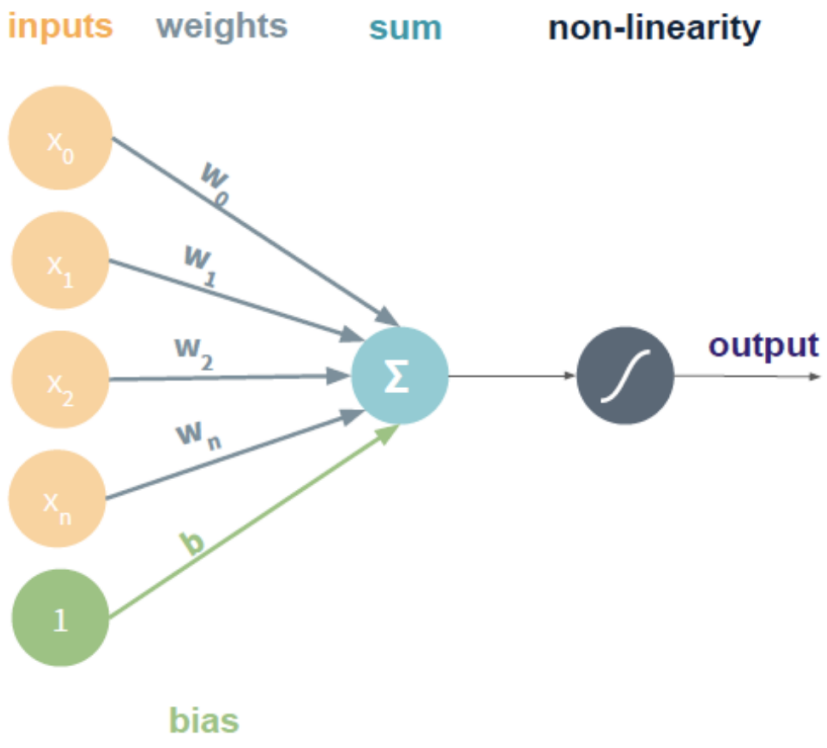


[2] Rosenblatt, F. (1958), The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain, Cornell Aeronautical Laboratory, Psychological Review, v65, No. 6.

The Perceptron

Activation Function

$$output = g((\sum_{i=0}^{n} x_i * w_i) + b)$$



Example of Forward Pass Calculation

(2*0.1) +

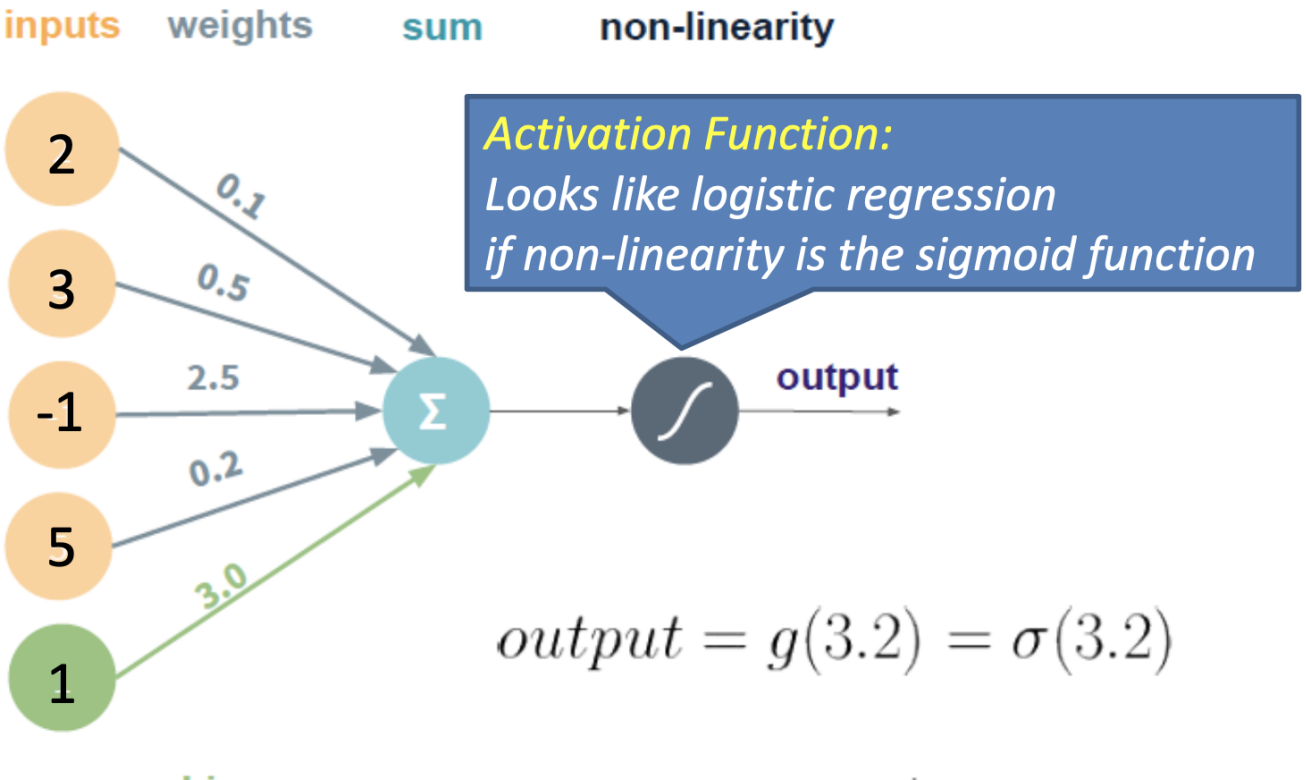
(3*0.5) +

(-1*2.5) +

(5*0.2) +

(1*3.0)

3.2



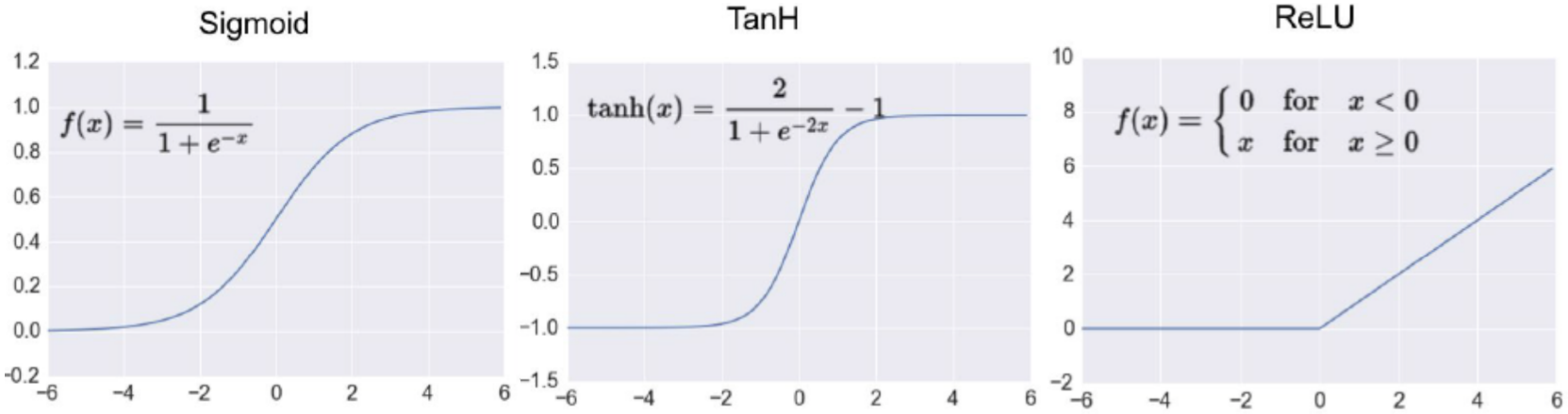
Activation Function:
Looks like logistic regression
if non-linearity is the sigmoid function

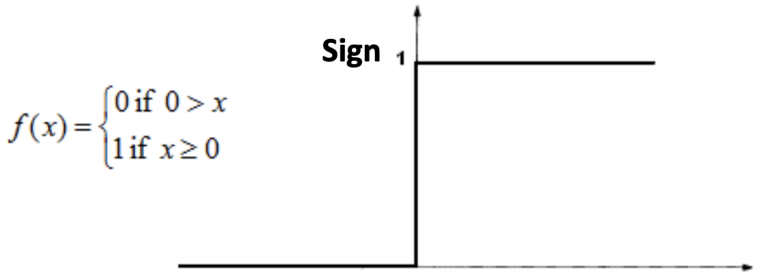
$$output = g(3.2) = \sigma(3.2)$$

$$= \frac{1}{(1 + e^{-3.2})} = 0.96$$

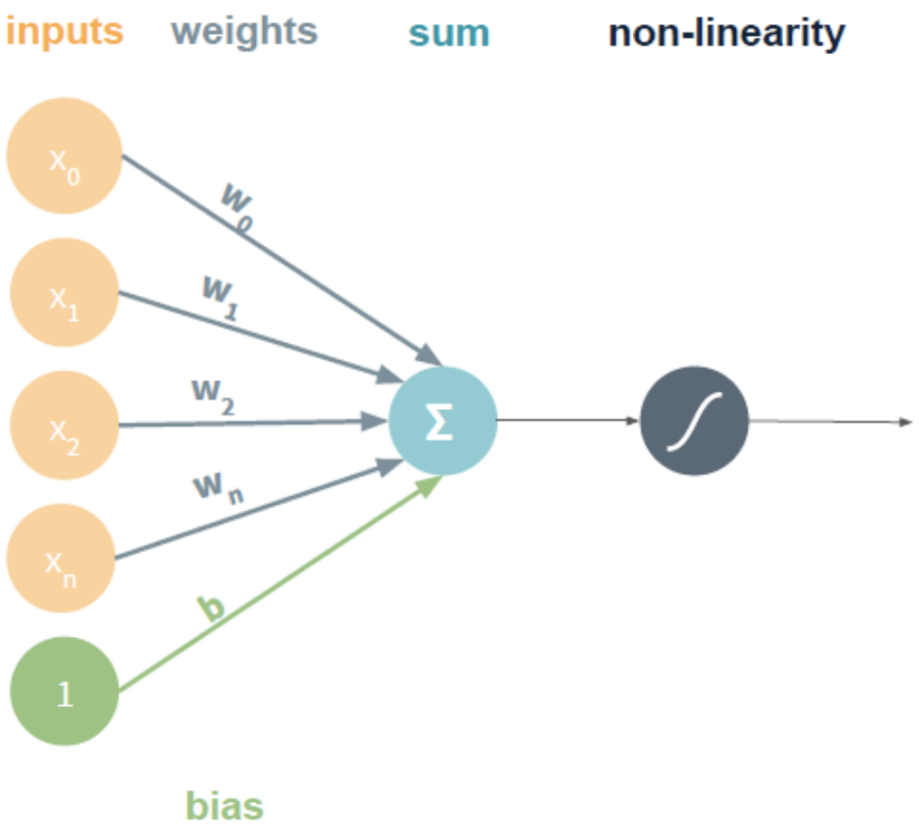
Common Activation Functions

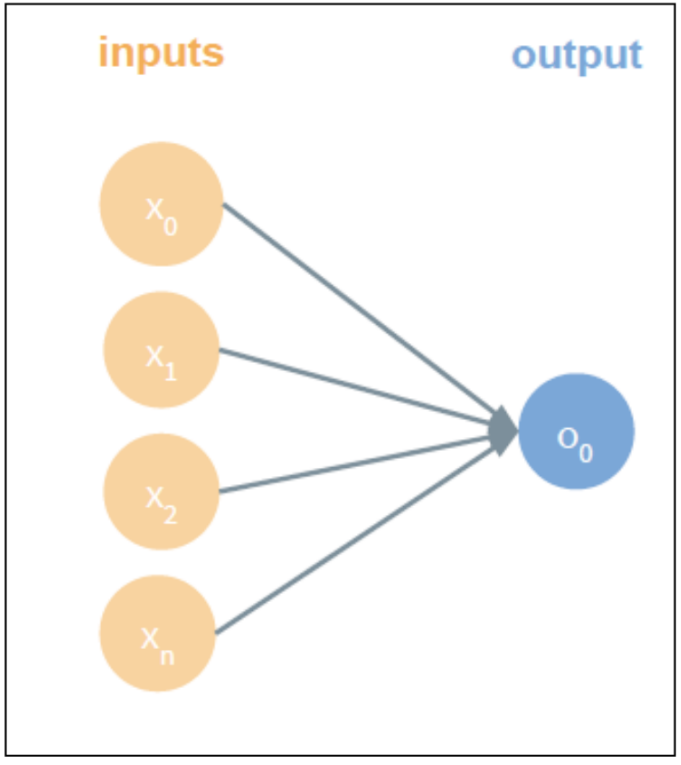
- Sigmoid, one of the first however *vanishing gradient issue*
- TanH, *zero-centered output* values but *vanishing gradient*
- ReLU, one of the most common today (good backprop)
- Sign (ony pedagocical)





Simplified Diagram

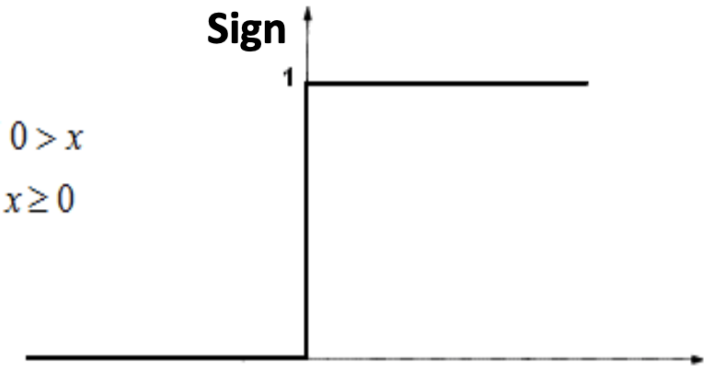




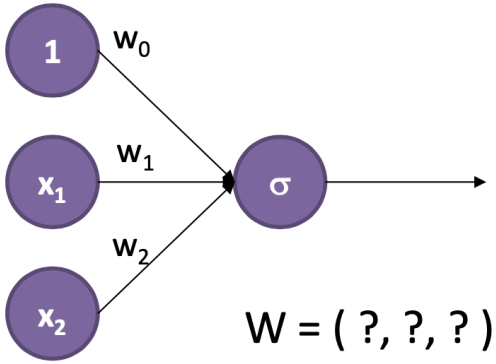
Computing OR

x_1	x_2	OR(x_1,x_2)
0	0	0
0	1	1
1	0	1
1	1	1

$$f(x) = \begin{cases} 0 \text{ if } 0 > x \\ 1 \text{ if } x \geq 0 \end{cases}$$



Activation function

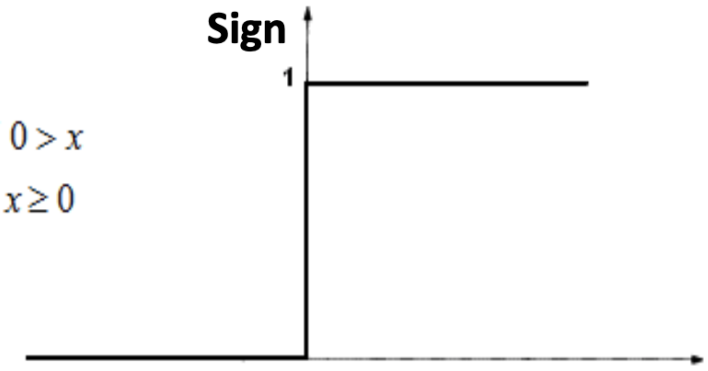


W = (?, ?, ?)

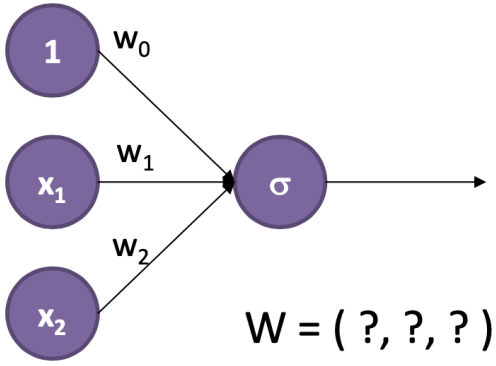
Computing OR

x_1	x_2	OR(x_1,x_2)
0	0	0
0	1	1
1	0	1
1	1	1

$$f(x) = \begin{cases} 0 \text{ if } 0 > x \\ 1 \text{ if } x \geq 0 \end{cases}$$



Activation function



W = (?, ?, ?)

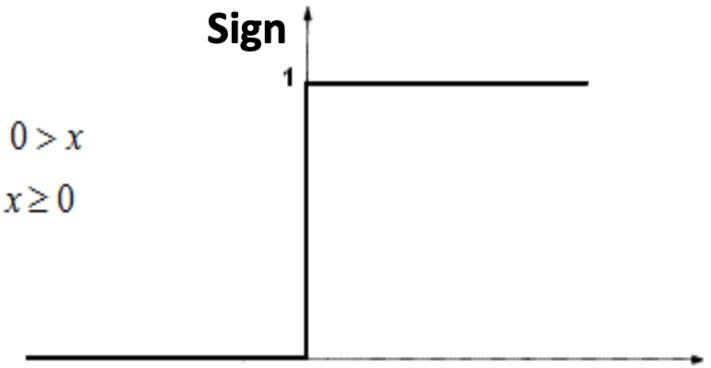
$$\boldsymbol{w} = (-50, 100, 100)$$

$$Sign(w_0 + \sum_{i=1}^{2} w_i x_i) = OR(x_1, x_2)$$

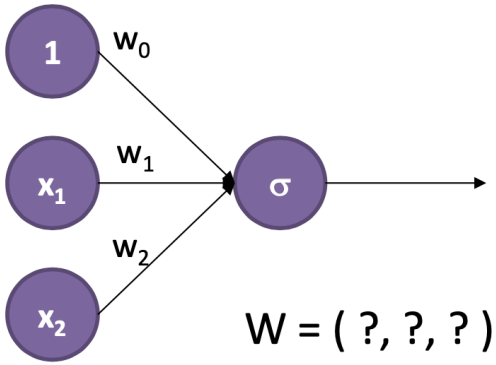
Computing AND

x_1	x_2	AND(x_1,x_2)
0	0	0
0	1	0
1	0	0
1	1	1

$$f(x) = \begin{cases} 0 \text{ if } 0 > x \\ 1 \text{ if } x \geq 0 \end{cases}$$



Activation function

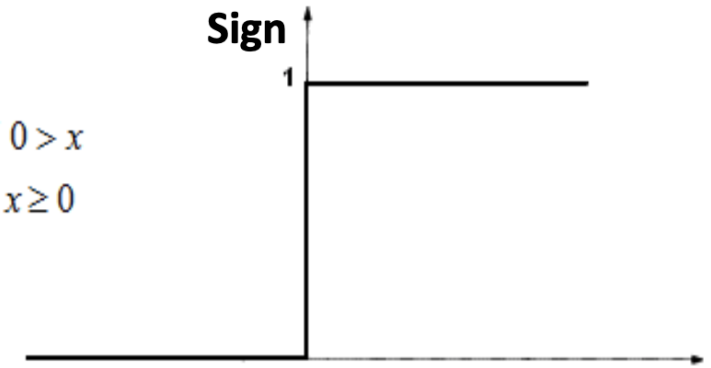


W = (?, ?, ?)

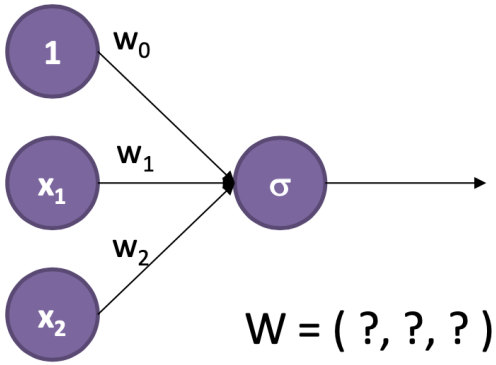
Computing AND

x_1	x_2	AND(x_1,x_2)
0	0	0
0	1	0
1	0	0
1	1	1

$$f(x) = \begin{cases} 0 \text{ if } 0 > x \\ 1 \text{ if } x \geq 0 \end{cases}$$



Activation function



W = (?, ?, ?)

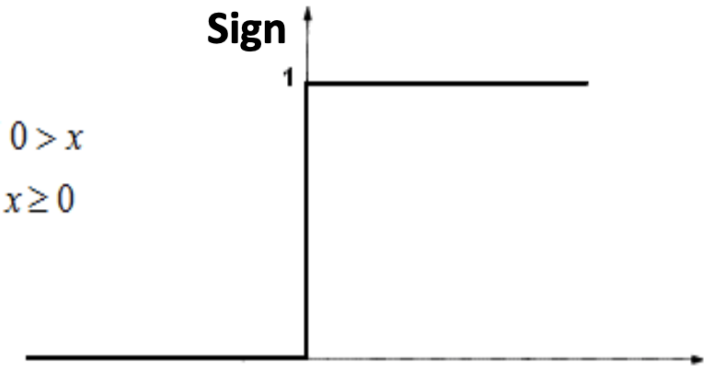
$$\boldsymbol{w} = (-150, 100, 100)$$

$$Sign(w_0 + \sum_{i=1}^{2} w_i x_i) = AND(x_1, x_2)$$

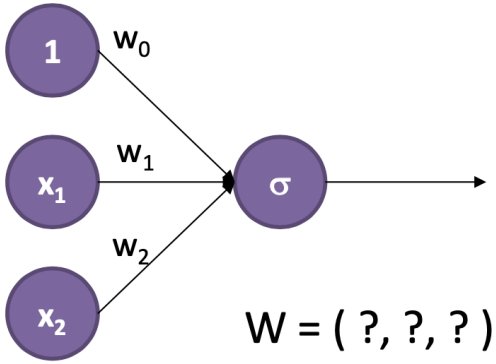
Computing XOR

x_1	x_2	x_1 XOR x_2
0	0	0
0	1	1
1	0	1
1	1	0

$$f(x) = \begin{cases} 0 \text{ if } 0 > x \\ 1 \text{ if } x \geq 0 \end{cases}$$



Activation function

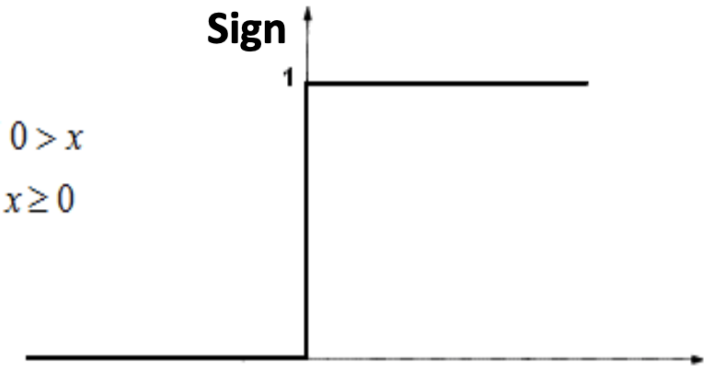


W = (?, ?, ?)

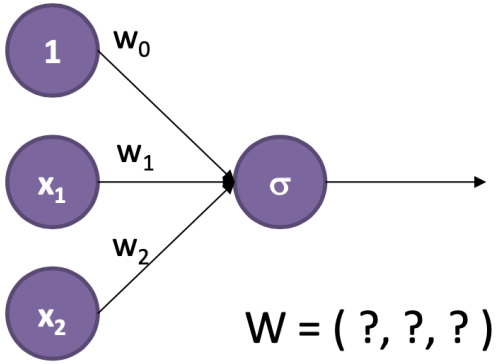
Computing XOR

x_1	x_2	x_1 XOR x_2
0	0	0
0	1	1
1	0	1
1	1	0

$$f(x) = \begin{cases} 0 \text{ if } 0 > x \\ 1 \text{ if } x \geq 0 \end{cases}$$



Activation function



W = (?, ?, ?)

Impossible!

Reminder: Linear Classifier

Decision Boundary: partitions the underlying vector space into sets, one per class. The classifier classifies all points in a same set to the same class, while points in different sets to different classes.

Decision boundary = hyperplane $\rightarrow$ **linear classifier**.

$\exists$ hyperplane separating 2 classes $\rightarrow$ points are **linearly separable**.

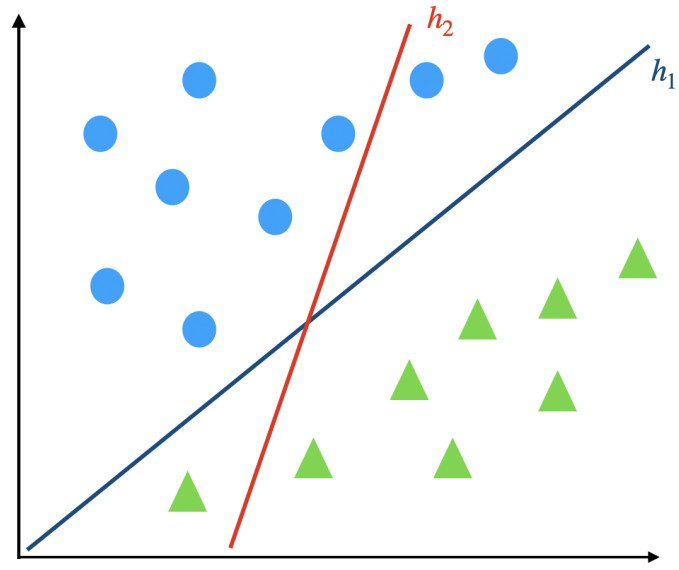


Figure: line h_1 separates the two classes $\rightarrow$ points are linearly separable.

Reminder: Linear Classifier

The perceptron (and logistic regression) are linear classifiers.

Decision boundary contains the points $\boldsymbol{x} \in \mathbb{R}^{\boldsymbol{d}}$ s.t.

$$\frac{1}{1+e^{-\boldsymbol{wx}}} = 0.5 \Leftrightarrow 1 = e^{-\boldsymbol{wx}} \Leftrightarrow 0 = -\sum_{i=1}^{d} w_i x_i$$

Same argument holds for any invertible activation function.

Observe that the output of the perceptron is not a linear function of its input, because the activation function is not linear.

XOR is not Linearly Separable

The perceptron is a linear class. but XOR is not linearly separable.

x_1	x_2	x_1 XOR x_2
0	0	0
0	1	1
1	0	1
1	1	0

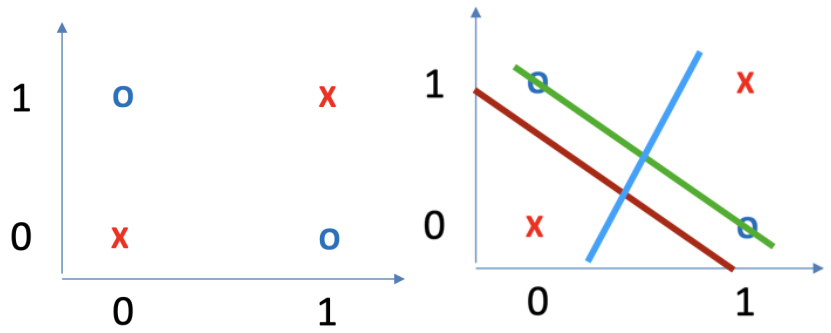
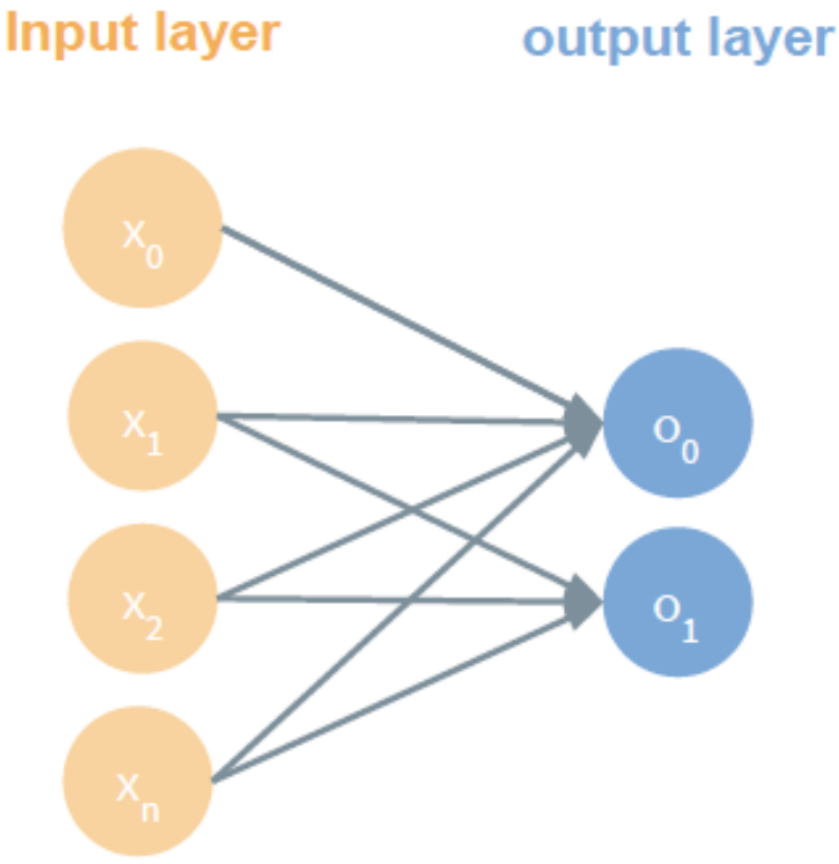
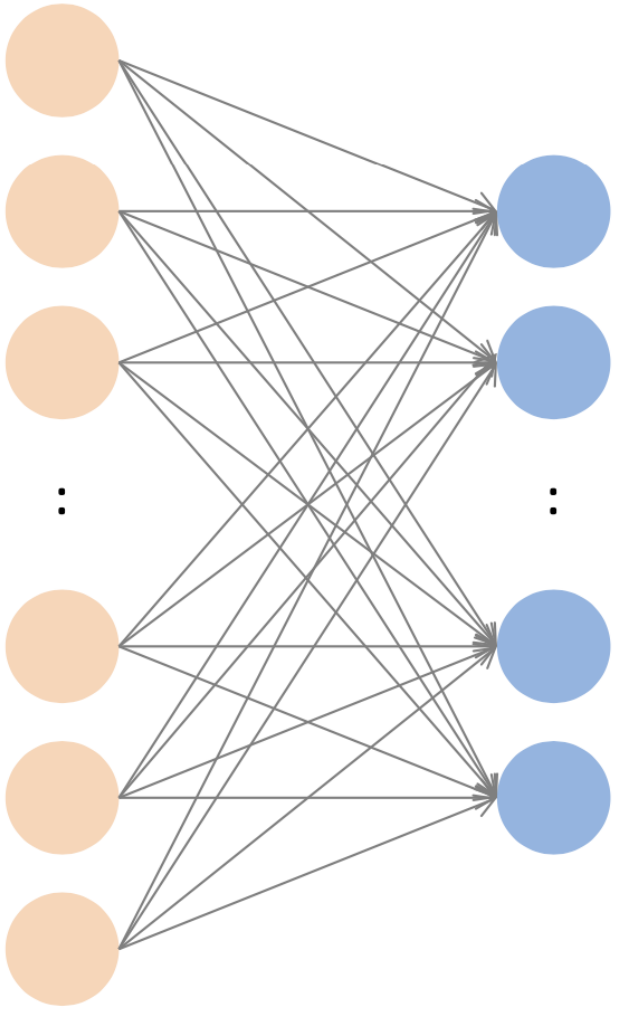


Figure: XOR: we cannot linearly separate the 0's from the 1's.

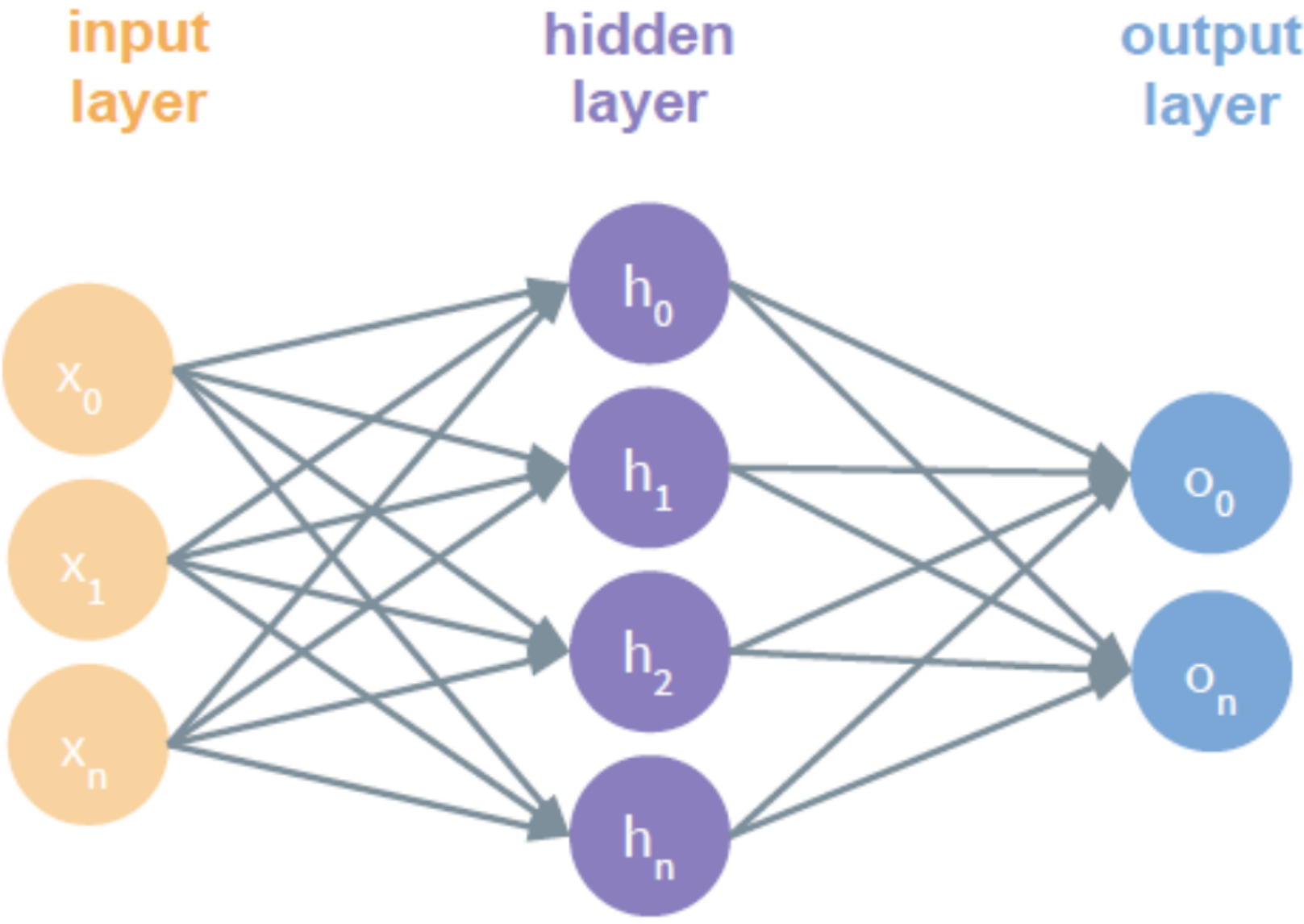
Hence, no way we can compute the XOR with one perceptron.

Multi-Output Perceptron

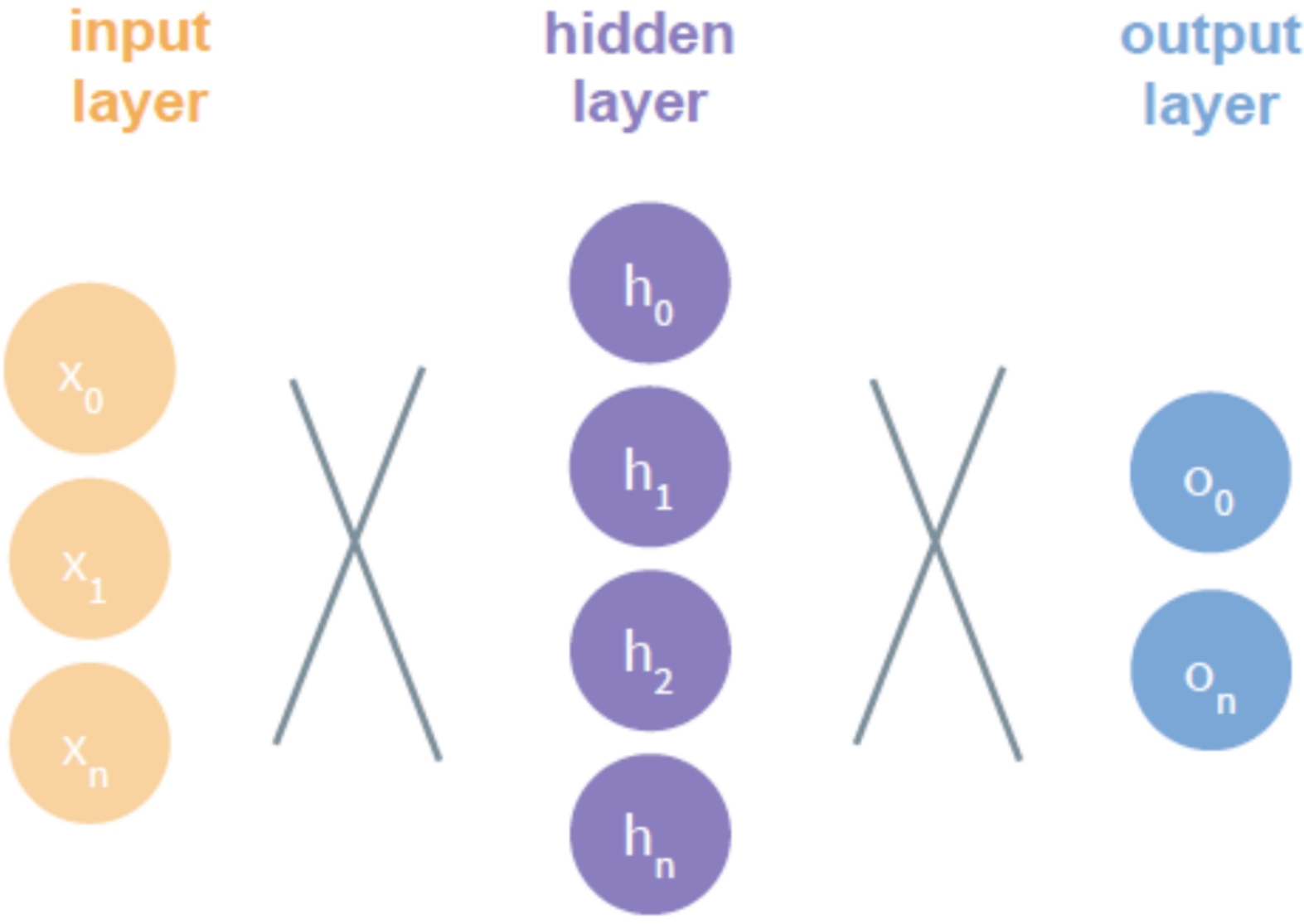




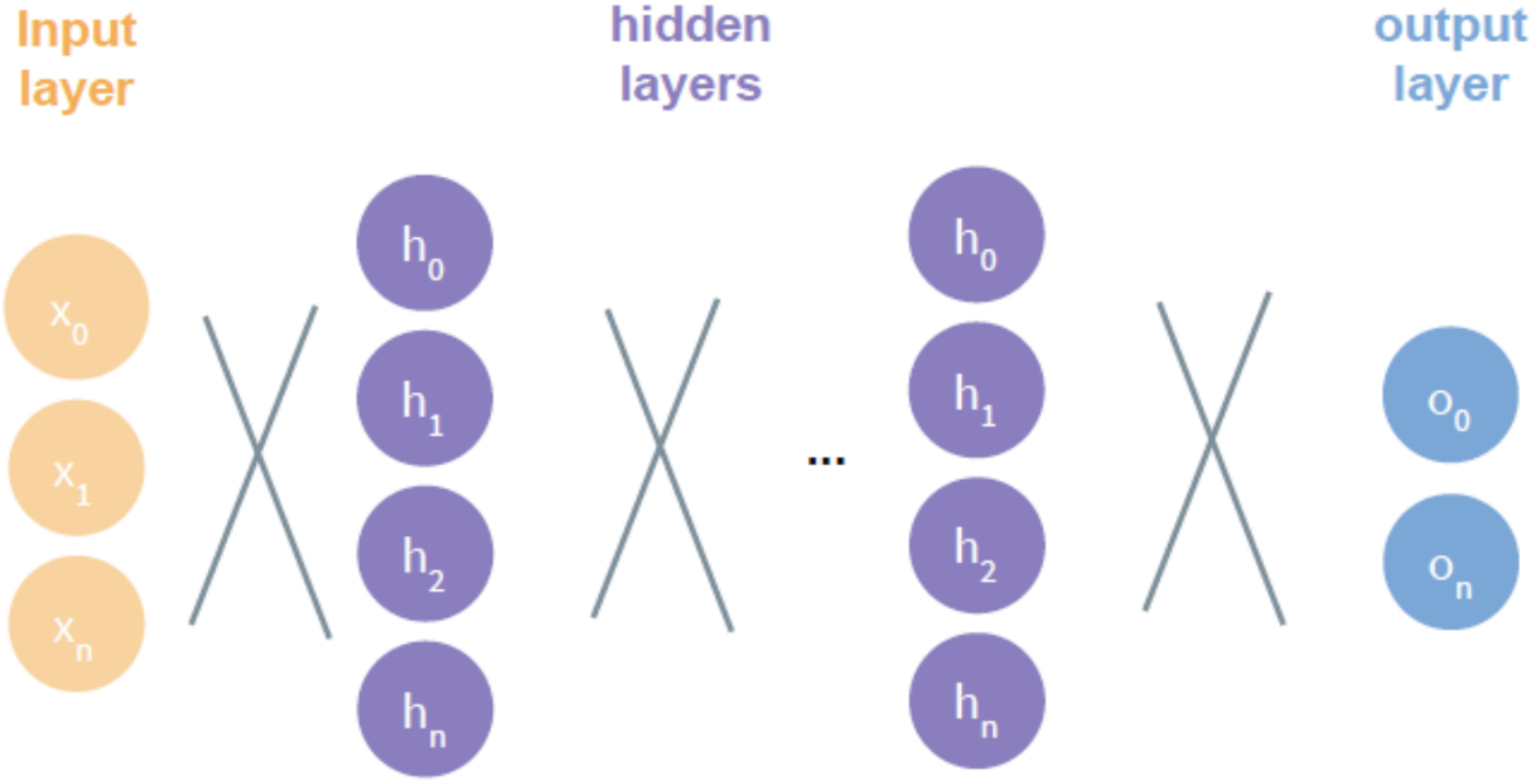
Multi-Layered Perceptron



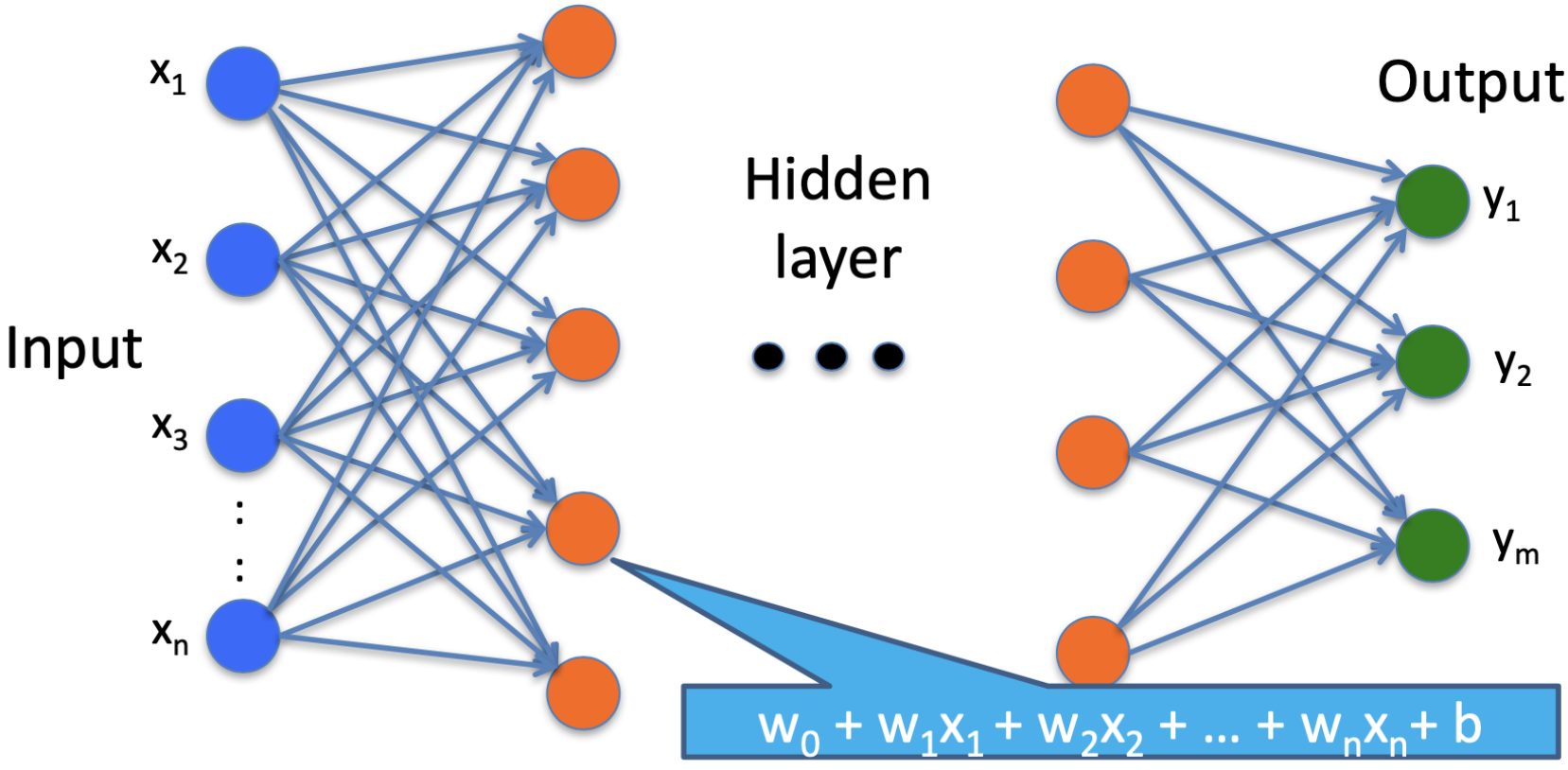
Multi-Layered Perceptron (simplified diagram)



Deep Neural Networks



ML Perceptron with Linear Activation Functions

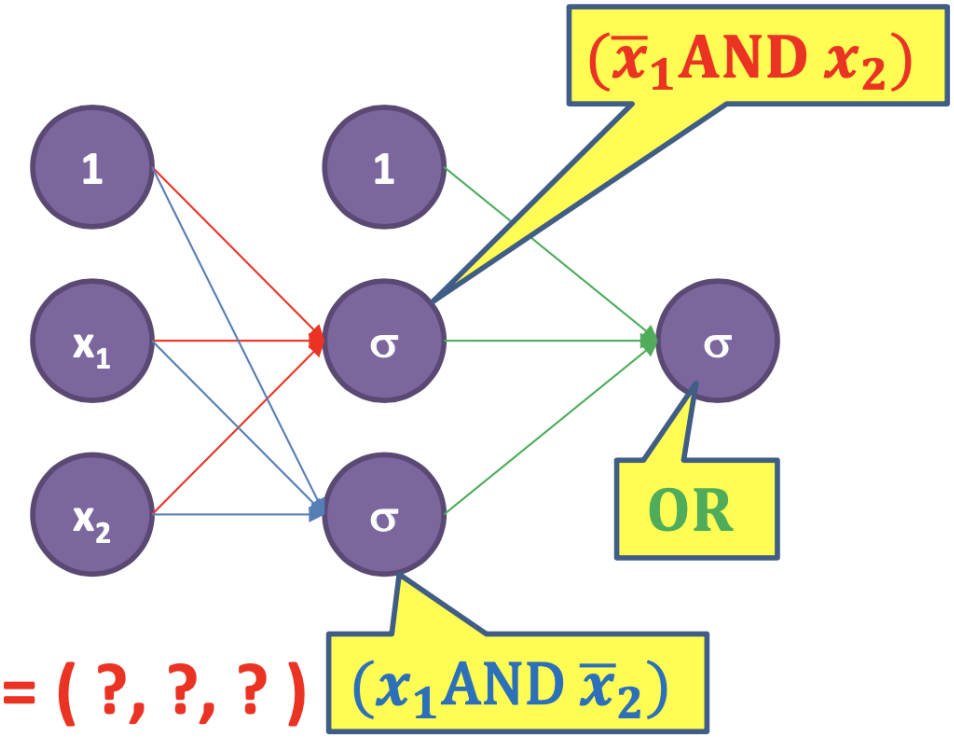


- if only linear activation function in hidden layers, no matter how many layers, still a linear classifier (e.g. XOR issue).
- use non-linear activation functions to increase model capacity.

Power of Layering Perceptrons (XOR)

$(\bar{x}_1 \text{ AND } x_2)$ OR $(x_1 \text{ AND } \bar{x}_2)$

x_1	x_2	
0	0	0
0	1	1
1	0	1
1	1	0



Remember

W_{AND} = (-150, 100, 100)

W_{OR} = (-50, 100, 100)

W = (?, ?, ?)

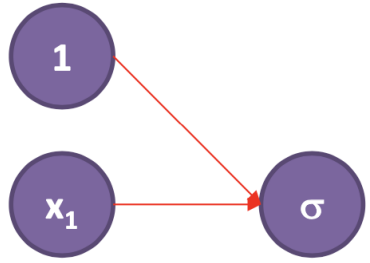
W = (?, ?, ?)

W = (?, ?, ?)

Power of Layering Perceptrons (NOT)

NOT

x_1	
0	1
1	0



$W_{NOT} = (100, -200)$

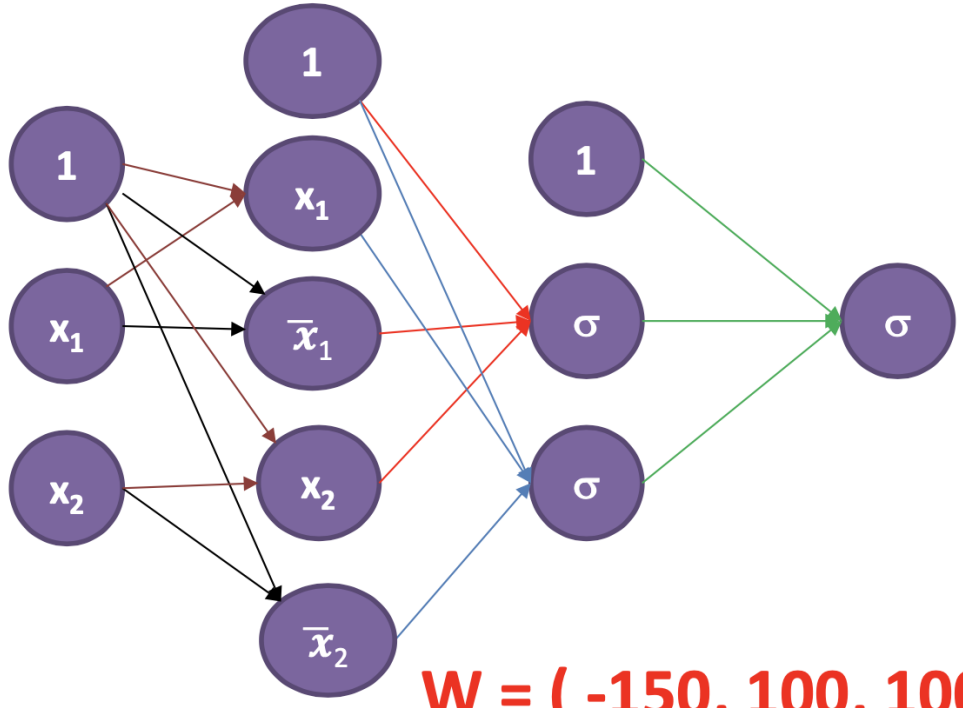
W = (?, ?)

- *logical completeness*: all boolean functions can be expressed
- NAND (NOT AND) or NOR (NOT OR) $\Longrightarrow$ logical complet.

Power of Layering Perceptrons (XOR)

$(\bar{x}_1 \text{ AND } x_2)$ OR $(x_1 \text{ AND } \bar{x}_2)$

x_1	x_2	
0	0	0
0	1	1
1	0	1
1	1	0



W_{AND} = (-150, 100, 100)

W_{OR} = (-50, 100, 100)

W_{NOT} = (100, -200)

W_{IND} = (-100, 200)

W = (-150, 100, 100)

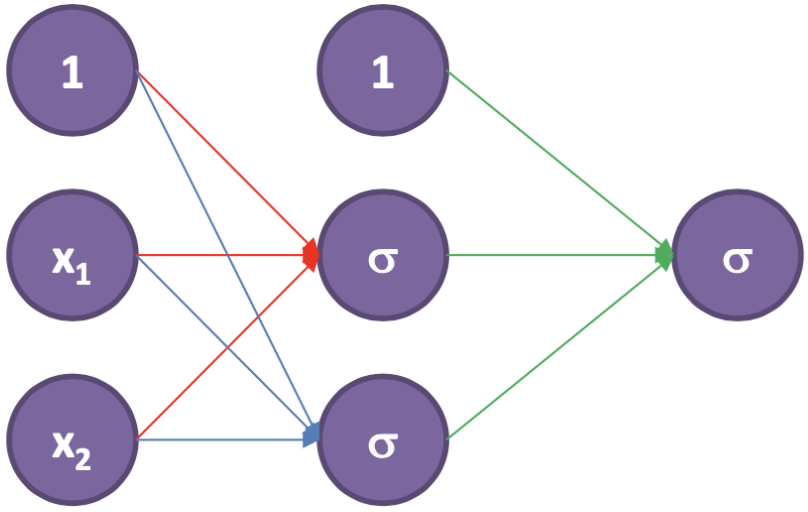
W = (-150. 100. 100)

W = (-50, 100, 100)

Power of Layering Perceptrons (XOR)

$$(\overline{x}_1 \text{ AND } x_2) \text{ OR } (x_1 \text{ AND } \overline{x}_2)$$

x_1	x_2	
0	0	0
0	1	1
1	0	1
1	1	0



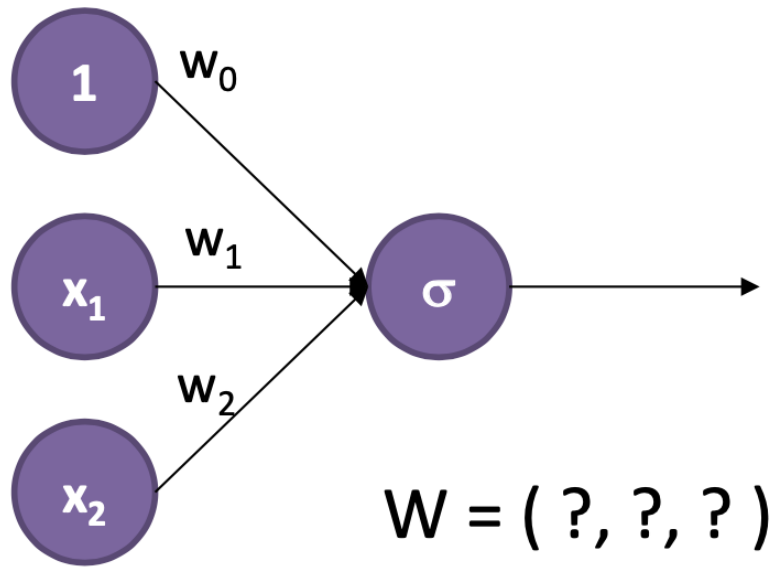
W_{AND} = (-150, 100, 100)
W_{OR} = (-50, 100, 100)
W_{NOT} = (100, -200)

W = (-150, -100, 200)
W = (-150, 200, -100)
W = (-50, 100, 100)

Exercise: NAND

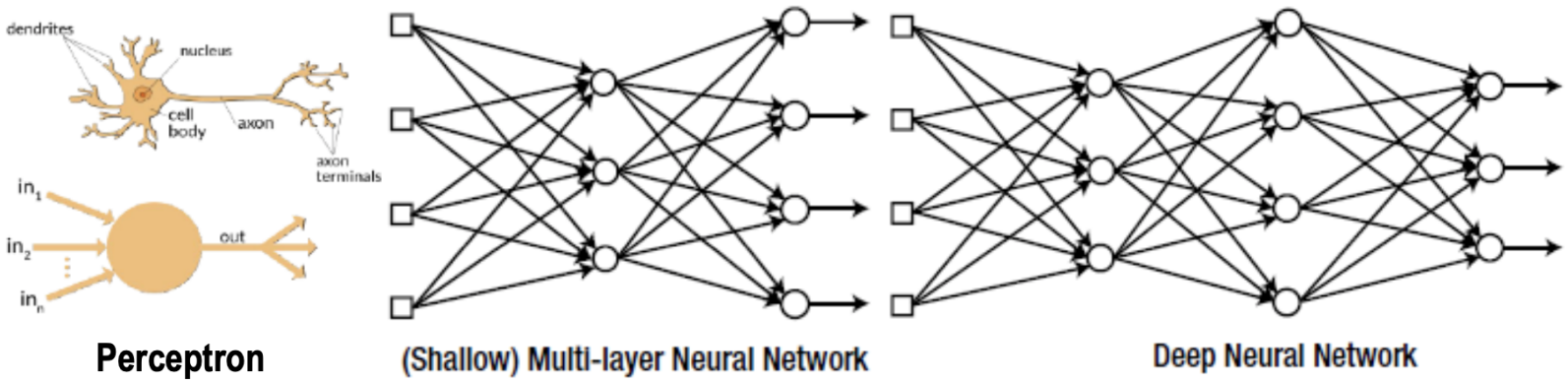
Develop a perceptron for the NAND gate.

h_1	h_2	
0	0	1
0	1	1
1	0	1
1	1	0

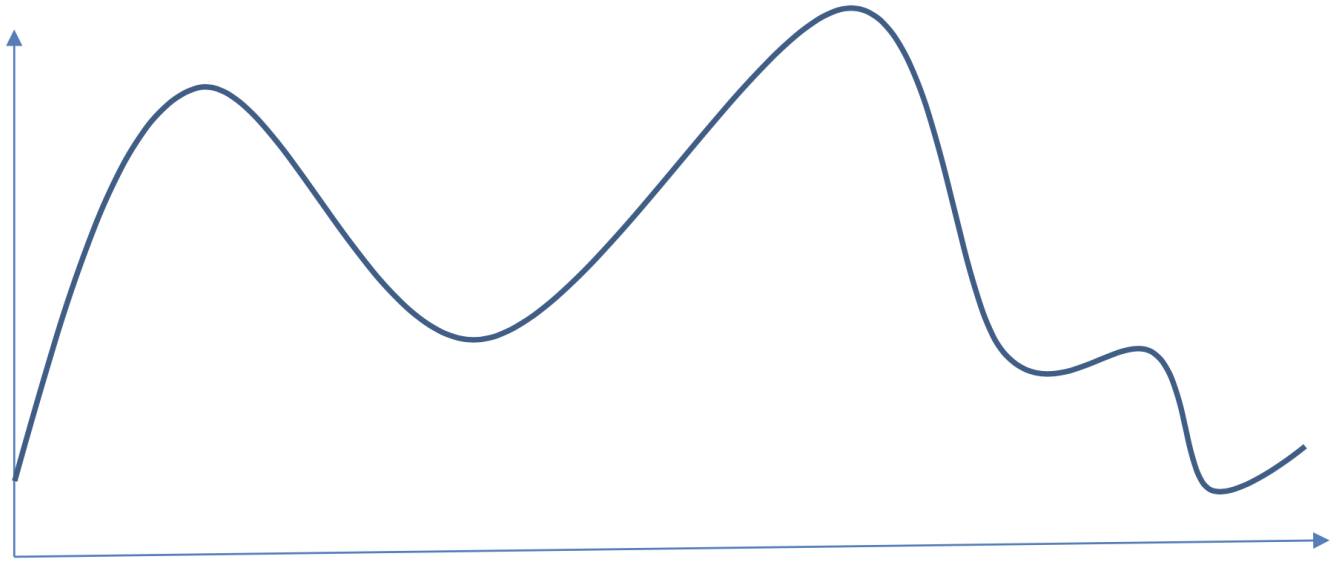


Historical Perspective

- Perceptrons (1958): quickly found out limitations (e.g. XOR)
- 1980s multi-layer NNs can approximate any function
- mid-2000s: Deep NN and the power of layers



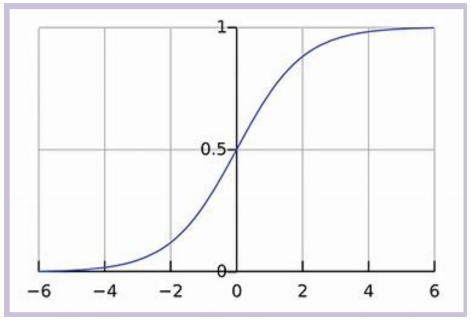
Can we approximate any function with NNs?



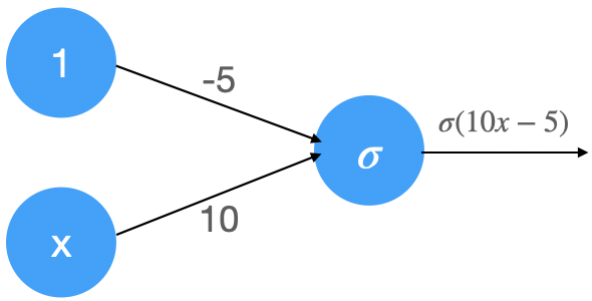
Universality Theorem: A visual Proof

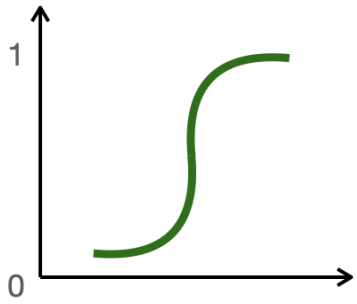
Consider the sigmoid activation function $\sigma : \mathbb{R} \mapsto (0, 1)$

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

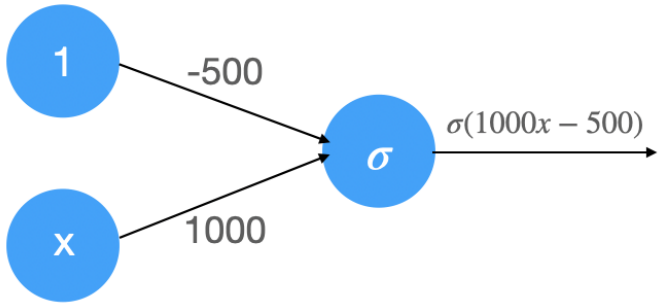


For example with weight 10 and bias -5 we obtain:

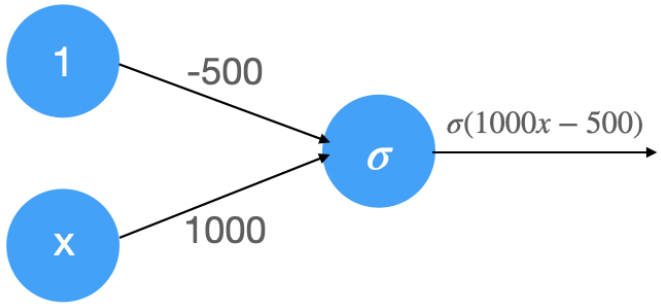


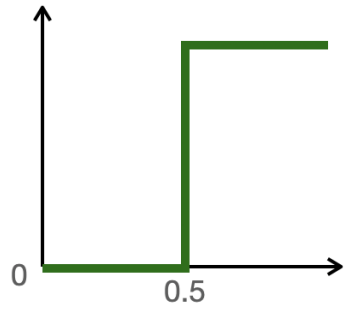


Universality Theorem: A visual Proof

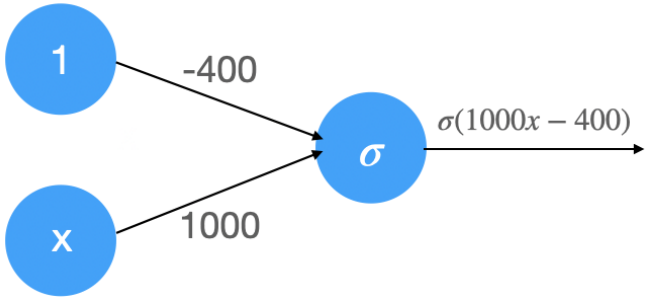


Universality Theorem: A visual Proof

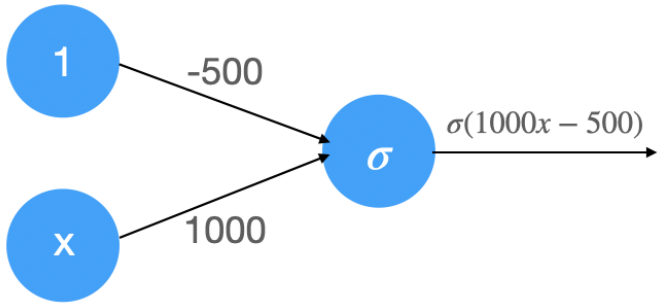


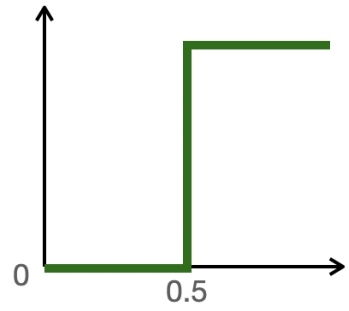


"Large" weights $\Longrightarrow$ sigmoid can be approx. by a step function.

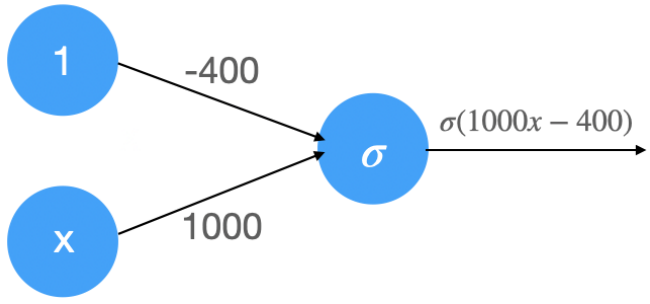


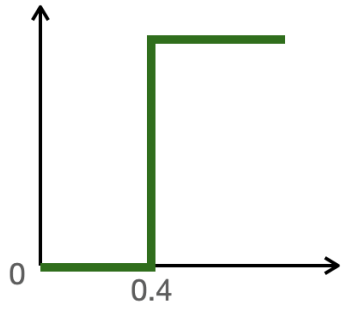
Universality Theorem: A visual Proof





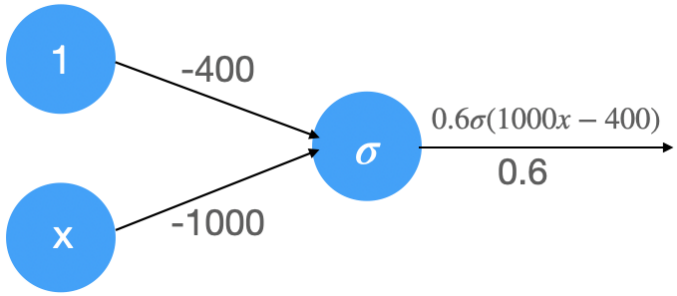
"Large" weights $\Longrightarrow$ sigmoid can be approx. by a step function.



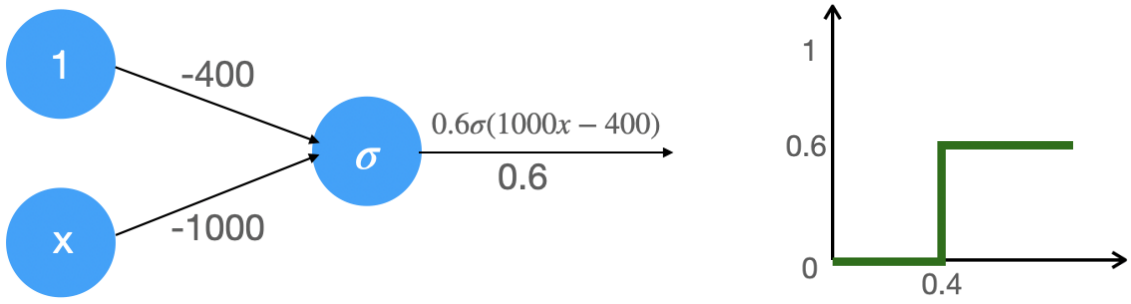


By changing bias, we control the "width" of the step function.

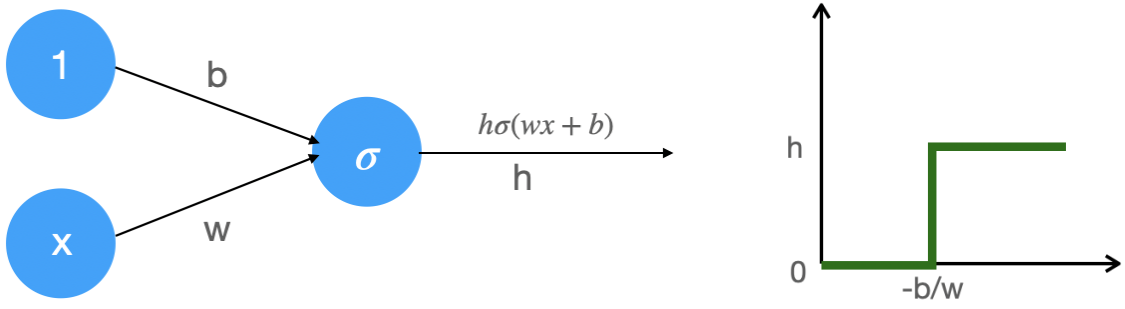
Universality Theorem: A visual Proof



Universality Theorem: A visual Proof



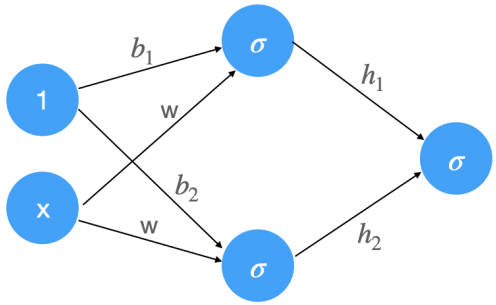
Output weight affects the "height" of the step function.



b, w, h allow us to control width and height of the step function.

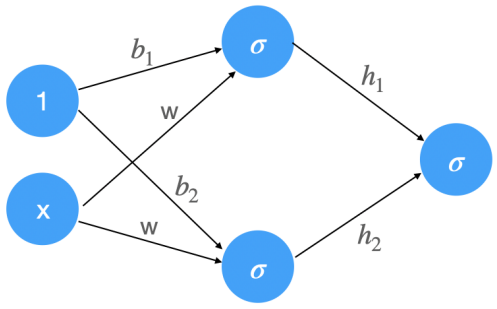
Universality Theorem: A visual Proof

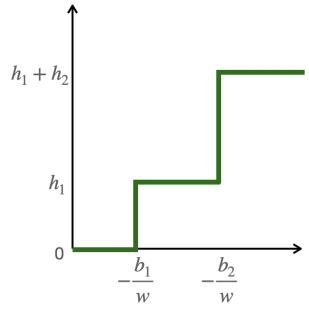
What happens if we combine NNs together?



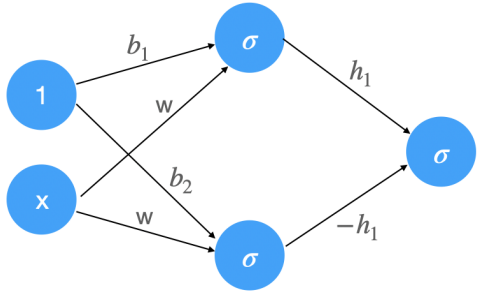
Universality Theorem: A visual Proof

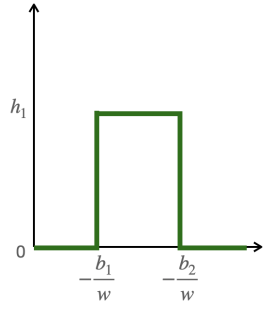
What happens if we combine NNs together?





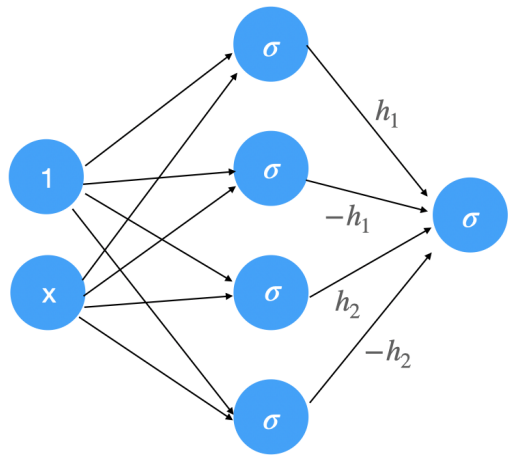
Hence, we can obtain a module...

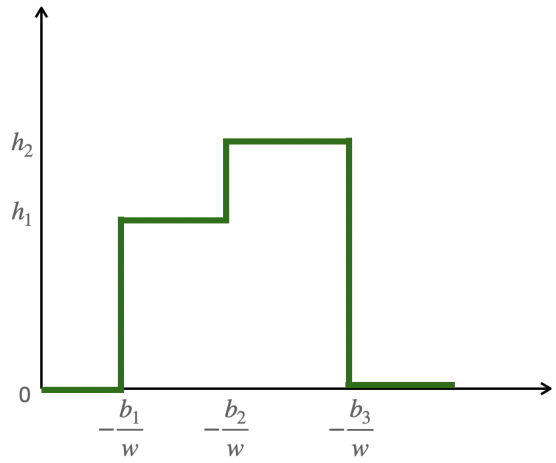




Universality Theorem: A visual Proof

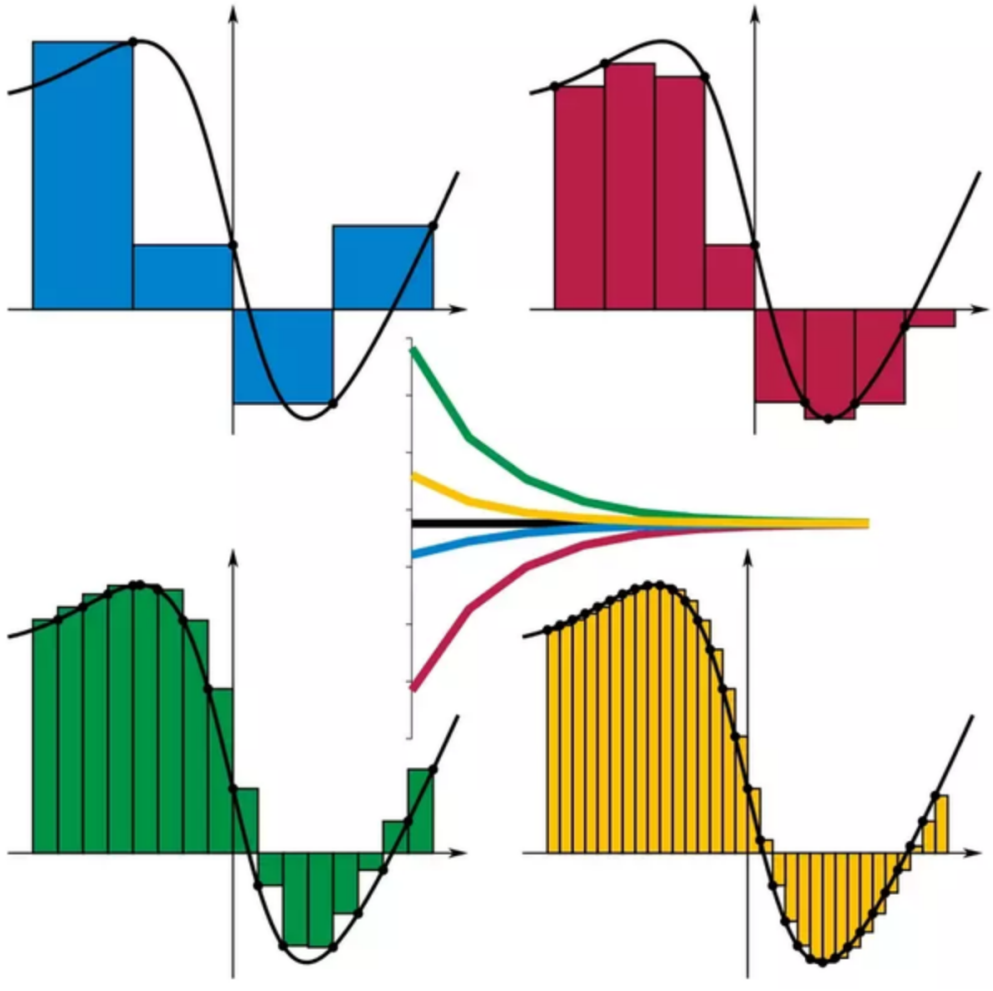
... that can be easily combined with other modules....





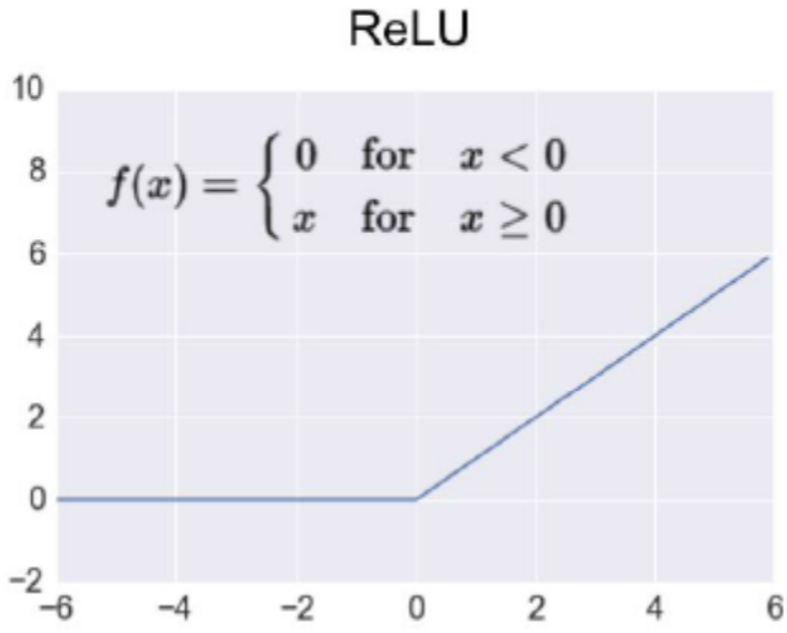
Can we approximate any function with NNs?

... so as to approximate any function, arbitrarily well.



What about ReLU?

$$ReLU(x) = \max(0, x)$$



Note: ReLu is not linear!

What about ReLU?

Rectangle in $[a, b]$ with height h via linear combinations of ReLU:

$$\frac{h}{\epsilon} * (ReLU(x - a) - ReLU(x - a - \epsilon) + ReLU(x - b - \epsilon) - ReLU(x - b))$$

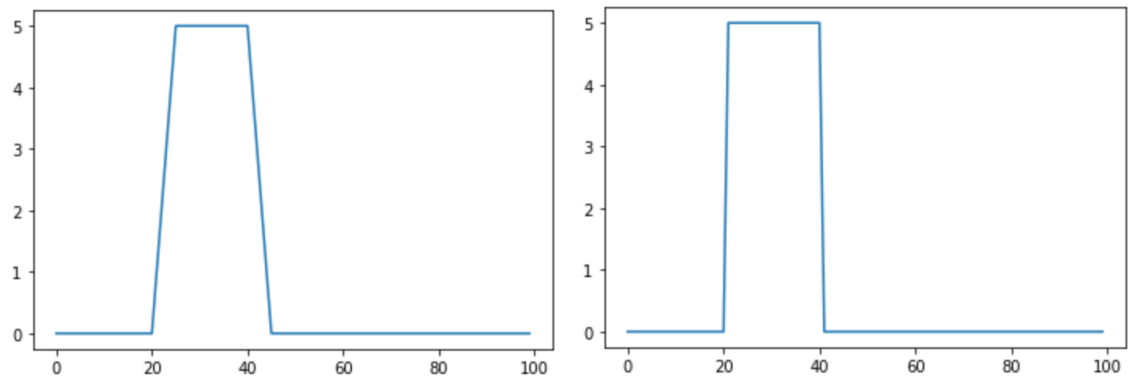
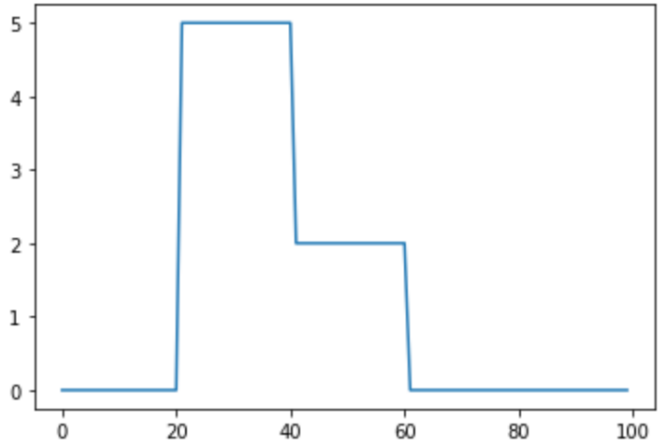


Figure: Plot of the function above with large (left) and smaller (right) ϵ

What about ReLU?

Again we get a module that can be combined with other modules...




```
import matplotlib.pyplot as plt

h1=5
a=20
b=40
eps=0.1
c=40
d=60
h2=2


def rect(x):
    return h1/eps*(max(0,x-a)-max(0,x-a-eps)+max(0,x-b-eps)-max(0,x-b))

def rects(x):
    return (h1/eps*(max(0,x-a)-max(0,x-a-eps)+max(0,x-b-eps)-max(0,x-b)) +
    h2/eps*(max(0,x-c)-max(0,x-c-eps)+max(0,x-d-eps)-max(0,x-d)))


l=list(range(0,100))
y=[rect(x) for x in l]
plt.plot(l,y)
```

Universality Theorem: Other Cases

- holds for all the activation functions considered so far and others (nor for linear functions). If the activation function is continuous, bounded and non-constant then it is OK [3].
- in our example we considered large width (max number of neurons per layer), recent studies show the theorem to hold even for width-bounded networks...

[3] K. Hornik, Approximation Capabilities of Multilayer Feedforward Networks, Neural Networks, 1991

Universality theorem for width-bounded ReLU networks [4]

Let the *width* be the maximum number of neurons per layer.

Theorem

For any Lebesgue-integrable function $f : \mathbb{R}^n \mapsto \mathbb{R}$, *any* $\epsilon > 0$, *there exists a fully-connected ReLU networks* $\mathcal{A}$ *with width* $\leq n + 4$, *such that the function* $F_{\mathcal{A}}$ *represented by* $\mathcal{A}$ *satisfies:*

$$\int_{\mathbb{R}^n} |f(x) - F_{\mathcal{A}(x)}| dx < \epsilon.$$

[4] Z. Lu et al. The Expressive Power of Neural Networks: A View from the Width. NIPS 2017.